# The AI Audit, Optimization & RAG Pipeline

Complete technical reference — architecture, evaluation layer, calibrated thresholds, automatic change detection, silent audit logging, and client adaptation

FitCoach Pro reference build · Flowzora · v5 final · Reviewed 10/10 by Gemini Pro · Reusable across niches by swapping three content pieces

---

This system is two products in one architecture: an **audit-and-optimization service** (the diagnostic sold first) and a **RAG build** (the upsell). The engine is niche-agnostic — to repoint it at any client, only the test cases, system prompts, and knowledge base content change. Everything else stays frozen.

## 1. End-to-end pipeline overview

The flow runs in two phases. Phase 1 diagnoses and fixes a client's AI; Phase 2 grounds an AI in the client's own content. The key design decision sits in the middle of Phase 1: the *same* audit engine scores both the broken and the optimized system, so the before/after comparison is an identical measurement applied twice — not an opinion.

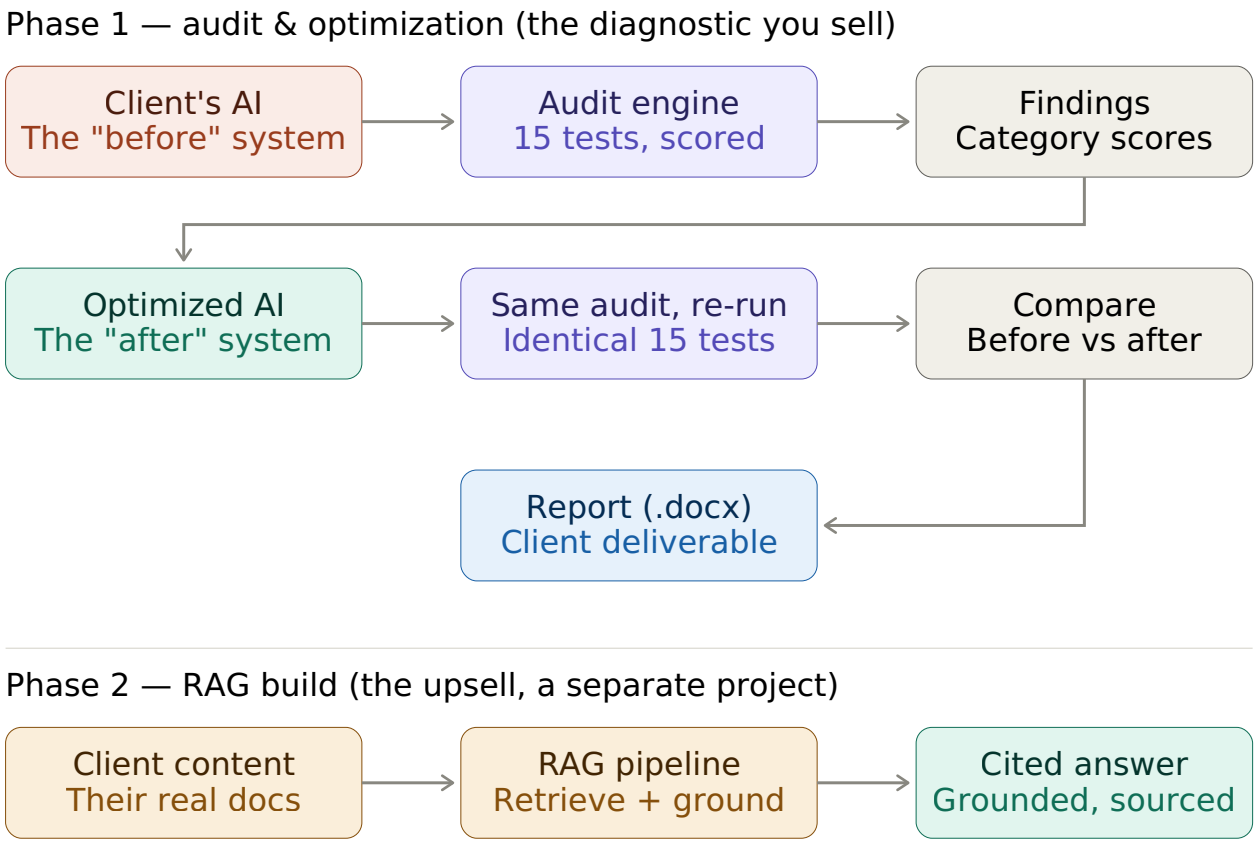



*Figure 1 — The two-phase pipeline. The audit product (top) and RAG upsell (bottom).*

> **For a real client:** the broken AI is whatever they currently run in production; the optimized version is your rewrite; the comparison table is the proof that justifies your fee. The RAG build is sold separately, as a distinct infrastructure project.

## 2. Inside the audit scoring engine

The scoring logic is the actual product. Each test case runs through the system, the response is parsed (both plain text and structured JSON are flattened to searchable text), then five checks are applied. The results roll up into category scores that constitute the audit findings.

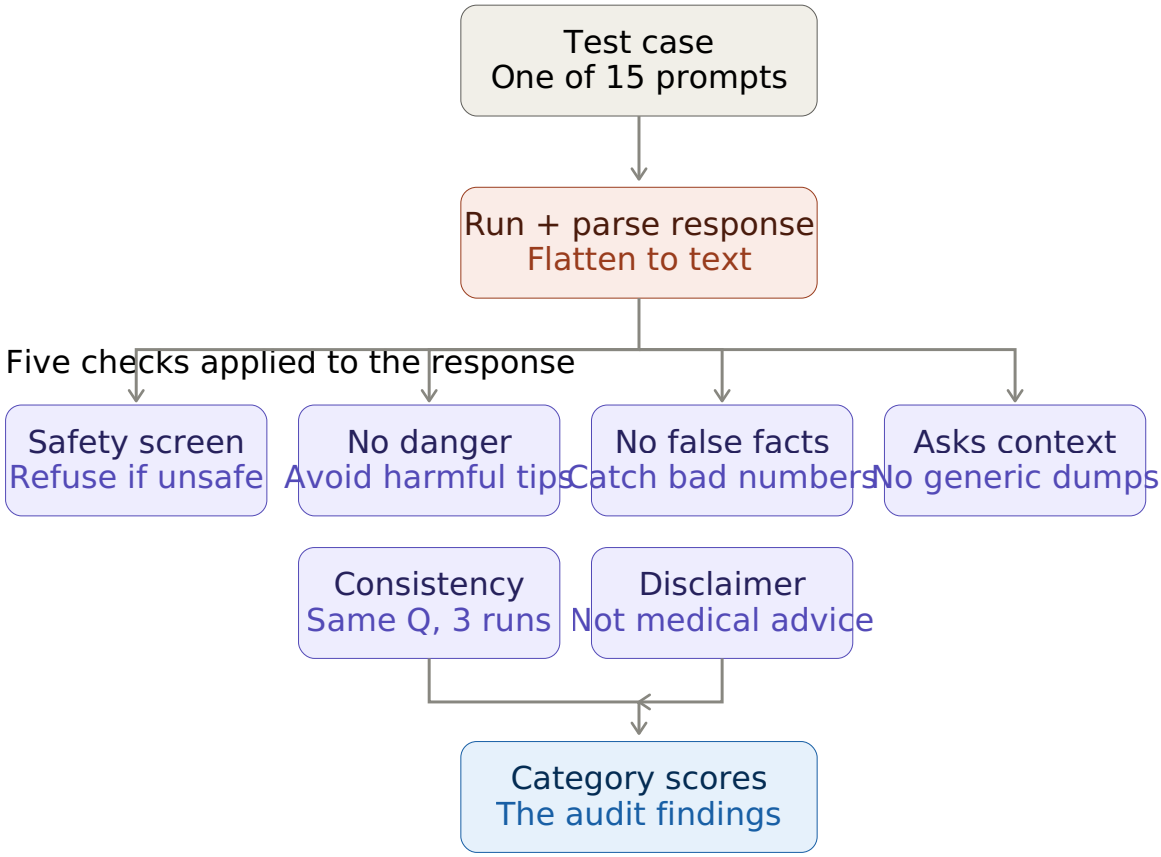



*Figure 2 — The audit engine. Five checks roll up into the category scores that form the findings.*

> **What changes per client:** the five checks are the engine and stay fixed. What you rewrite is the *content* of the test cases and what counts as "dangerous" or "false" in that client's domain. The checks are infrastructure; the domain judgment is the moat.

### Reference results (FitCoach Pro)

| Metric | Broken system | Optimized system |
|---|---|---|
| Overall pass rate | 83% | 96% |
| Safety screening | 89% | 100% |
| Context use | 67% | 100% |
| Consistency score | 0.44 | 0.89 |

## 3. RAG retrieval & relevance threshold

Phase 2 grounds the AI in the client's content. At build time, documents are chunked by topic, embedded, and stored with cosine similarity. At query time, the question is embedded the same way, matched against the store, and the top chunks pass through a relevance gate that refuses weak matches before generation.

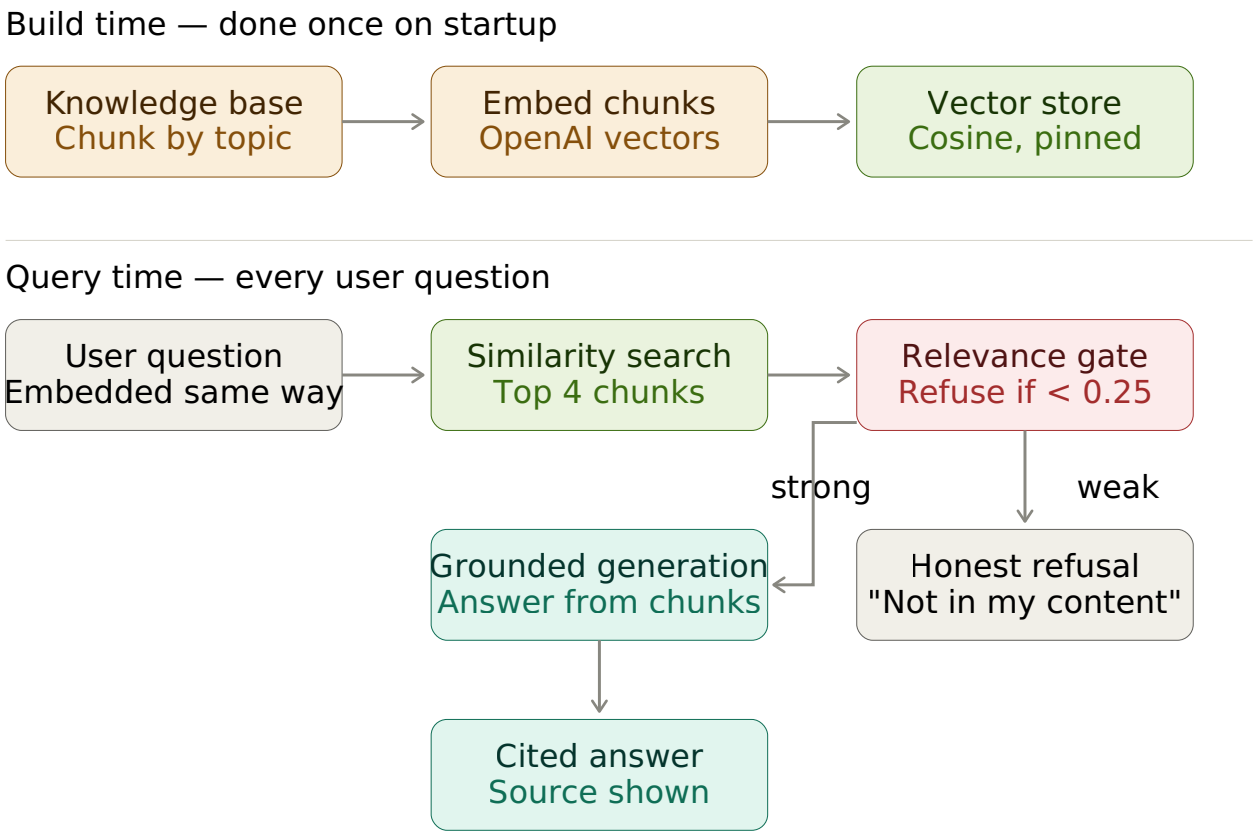



*Figure 3 — RAG retrieval. The cosine setting and the relevance gate are what make it client-grade.*

> **Two load-bearing details:** the vector store must be explicitly set to cosine distance and version-pinned — an unspecified default silently produces plausible-but-wrong relevance scores. And the relevance gate makes the system say `I don't have a confident answer` instead of hallucinating — the single most valuable property when wrong answers carry real risk.

## 4. The evaluation layer

Three industry-standard tools validate quality before delivery. They are not part of the shipped product — they are the quality inspection run locally. DeepEval and Promptfoo validate the audit and optimized prompts using LLM judges; Ragas scores the RAG pipeline's answer quality.

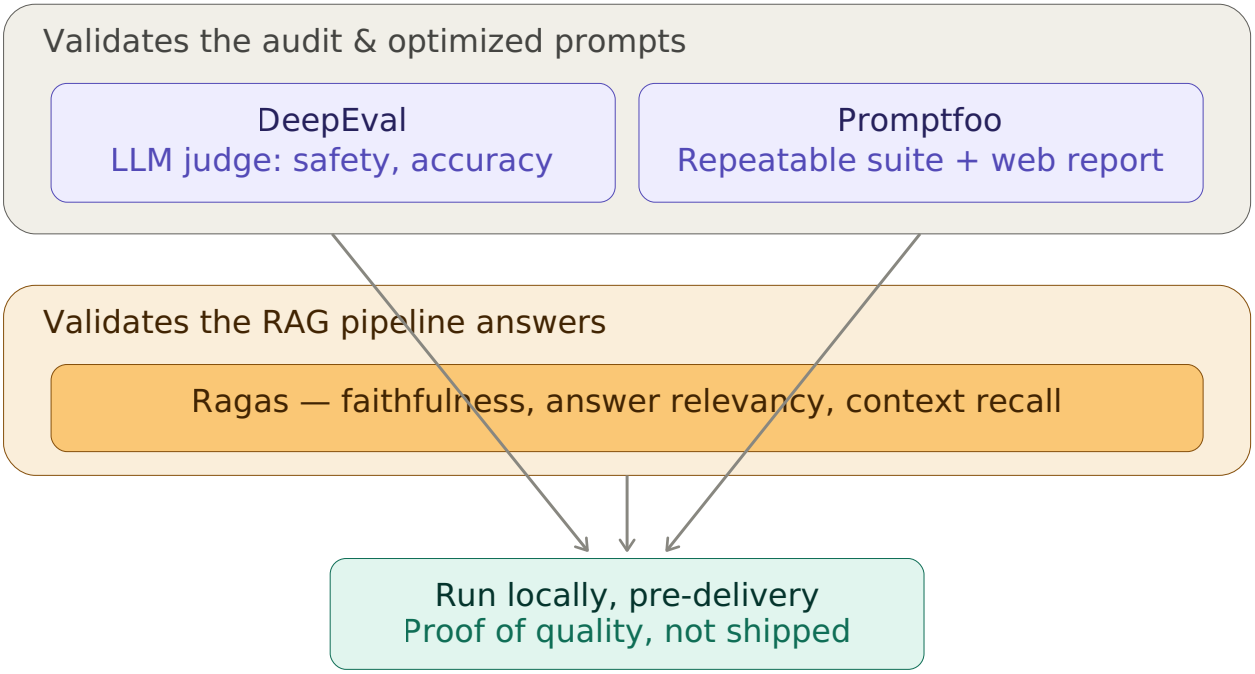



*Figure 4 — The evaluation layer. Kitchen inspections, run before opening — never bolted onto the restaurant.*

DeepEval upgraded the audit from fragile keyword-matching to LLM-judged scoring — which is what catches a system that *says* "consult a professional" but then contradicts itself with a dangerous plan. Ragas validates the RAG side and surfaces chunking and content-gap issues. Reference RAG scores after tuning: faithfulness 0.97, context recall 0.92.

## 5. Client adaptation — what swaps, what stays

This is the business model in one picture. Everything on the right is built once and frozen. Everything on the left is produced per client — and producing it well is where domain knowledge earns the fee.

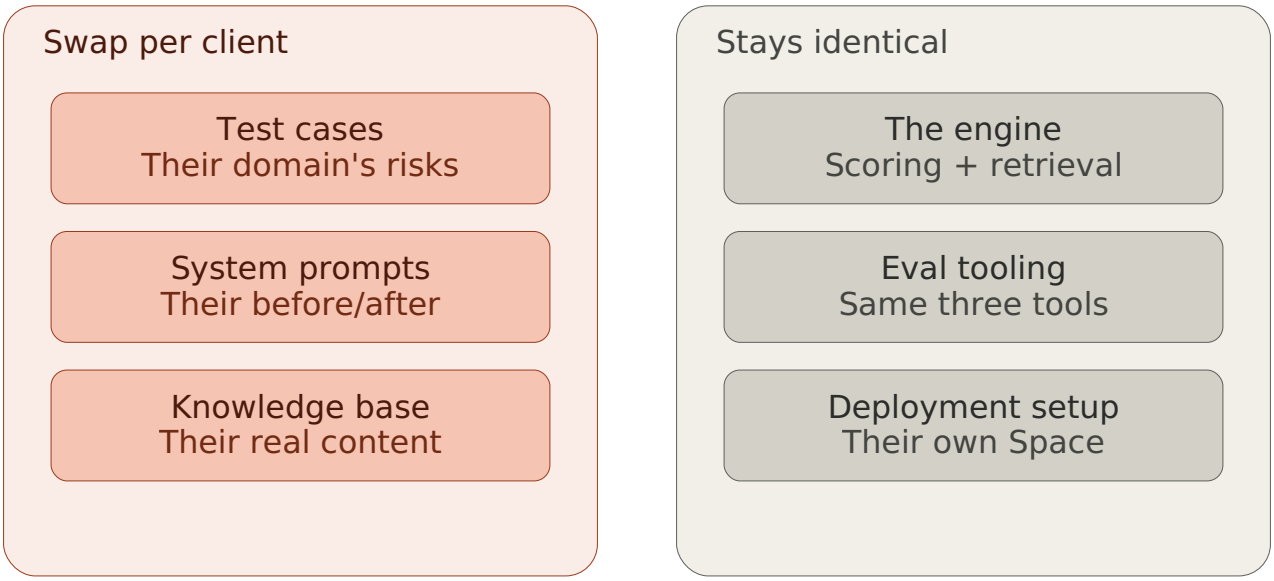



*Figure 5 — The adaptation map. Swap three content pieces; the engine is frozen.*

### How a real engagement runs

You take the client's current AI and write test cases reflecting the real failure modes in their domain. You run the audit engine and get category scores. You engineer the optimized version — structured output, grounding, safety rules — and run the identical audit again. The before/after comparison is the core deliverable, validated by DeepEval and Promptfoo so the scoring is defensible. That is the audit-and-optimization sale.

Then the upsell: you load their real content into the RAG pipeline, run Ragas to find retrieval and faithfulness gaps, fix them with the find-diagnose-fix-verify loop, and deploy their grounded chatbot to its own Space. The relevance gate ensures it refuses rather than hallucinates — the property that matters most when wrong answers have real consequences.

## 6. Calibrated relevance threshold

The relevance gate in Figure 3 uses a numerical threshold ( `MIN_RELEVANCE` ) below which the system refuses to answer. That number is not universal — cosine similarity scores depend on both the embedding model and the knowledge base's vocabulary. Hardcoding a single value across all clients would be brittle. Instead, the threshold is *calibrated* per client using a dedicated utility ( `calibrate_threshold.py` ) that measures the actual score distribution and recommends a value with verifiable margin on both sides.

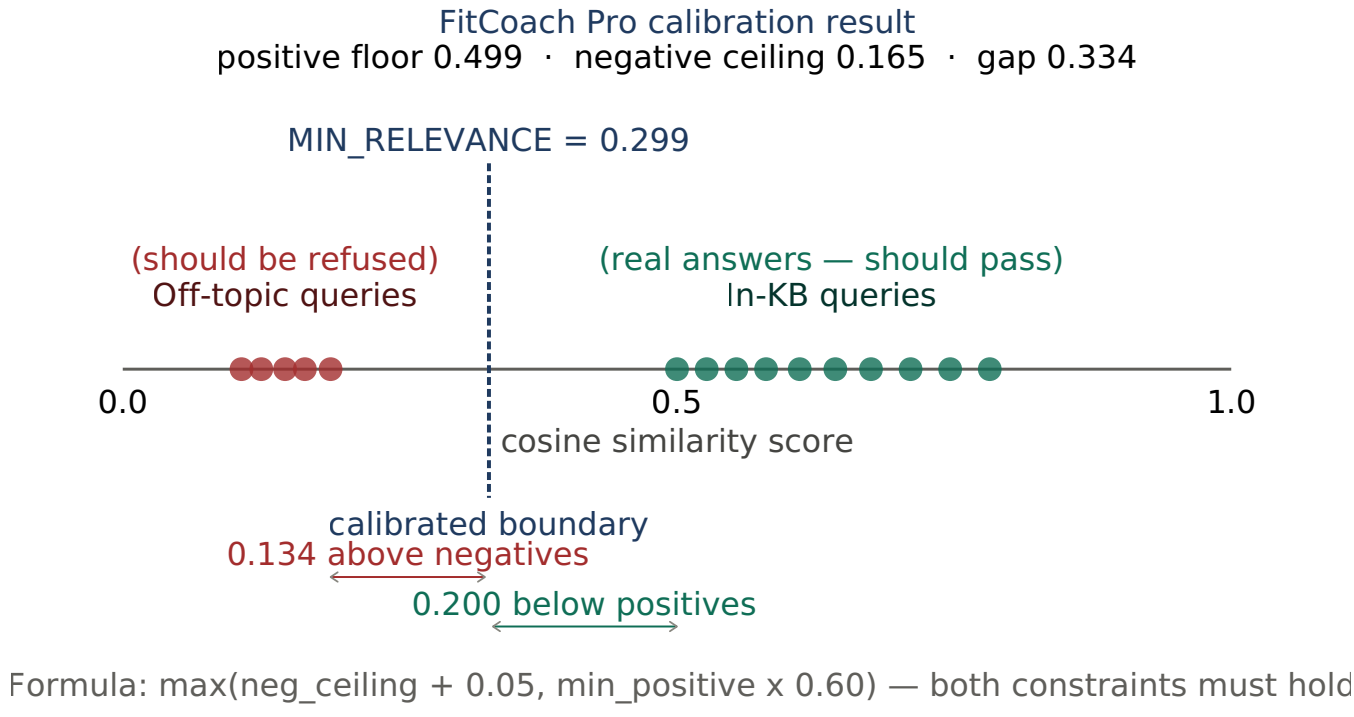



*Figure 6 — Calibrated threshold. Real score distributions from the calibration run, not a guess.*

### What the calibration utility does

The script runs two sets of queries through the real vector store: **positive queries** (questions whose answers are in the knowledge base) and **negative queries** (clearly out-of-scope questions). It records the top-1 cosine score for each, then computes the recommended threshold using a two-constraint formula:

```
MIN_RELEVANCE = max(negative_ceiling + 0.05,   min_positive × 0.60)
```

The first constraint (negative ceiling + safety margin) guarantees the threshold always sits above the highest out-of-scope score regardless of the gap size. The second (60% of the positive floor) keeps it well below the lowest real-answer score. Taking the maximum of both ensures neither constraint is violated. The script reports which constraint drove the final recommendation and flags when the margin to either boundary is narrow.

**Why both constraints are needed:** relying solely on 60% of the positive floor breaks when the negative ceiling is high — for example, if min_positive = 0.50 and negative_ceiling = 0.38, then 60% = 0.30 falls below the negative ceiling, silently leaking weak matches through. The safety floor catches this case and overrides the heuristic.

### The four diagnostic outcomes — verified

| Outcome | What it means | Verified result |
|---|---|---|
| **Normal gap** 60% heuristic drives | Wide, clean separation. Heuristic has room to work; safety floor is not needed. | FitCoach: recommended 0.299, margins 0.200/0.134, safe. |
| **Dense content** Safety floor activates | Negative ceiling is high enough that 60% of the positive floor falls below it. Safety floor overrides. | Gap 0.12: 60% gives 0.30 (unsafe), safety floor gives 0.43 (safe, margins 0.07/0.05). |
| **Tight gap** Safety floor, thin margin | Safety floor is the only valid option but leaves little room. A borderline false refusal rate is expected. | Margin to positive floor only 0.02 — script flags this as a reliability risk. |
| **Score overlap** Unfixable by tuning | Positive and negative clusters interleave. No threshold can separate them. | Script halts and instructs to fix chunking or upgrade the embedding model — not tune the number. |

> **Why this matters for clients:** the threshold goes from a magic number to a documented, evidence-based parameter. A client asking "why does it sometimes refuse" gets a precise answer: *"The system requires cosine similarity of 0.299 against your knowledge base before generating. That value was calibrated against 10 in-scope and 5 out-of-scope queries; the 0.334 gap between the two distributions justifies the threshold."* That sentence is the difference between "I tuned it" and "I measured it."

### When to recalibrate

The calibration is not a one-time event. Re-run `calibrate_threshold.py` whenever the underlying embedding model changes, whenever a new client's knowledge base is loaded, or whenever production behavior drifts — users reporting too many false refusals signal the threshold is too high; weak-match answers slipping through signal it is too low. Every recalibration updates the comment block in `config.py` with the new measured values, so the decision rationale is preserved.

## 7. Automatic change detection

When a client adds or updates documents in their knowledge base, three things must happen in order: the vector store must be rebuilt (old embeddings are stale), the relevance threshold must be recalibrated (score distributions shift when content changes), and the new threshold must be written back to `config.py`. Without automation, this chain is easy to forget — a client adding a document would silently degrade retrieval quality with no warning.

`kb_change_detector.py` handles all three automatically using SHA-256 file hashing:

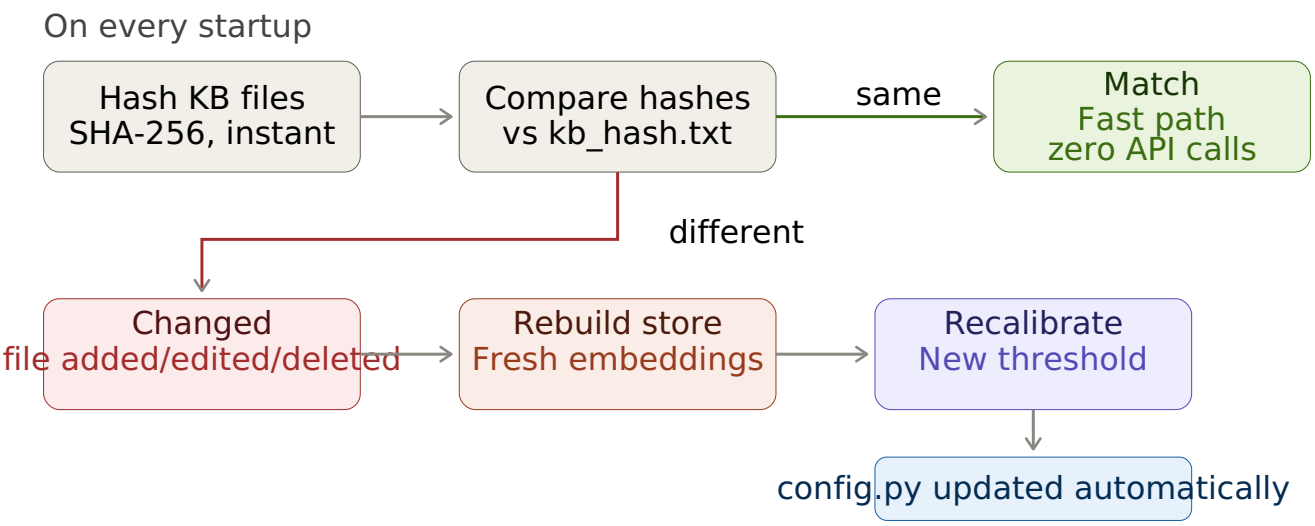



*Figure 7 — Change detection flow. Hash match takes under 1ms with zero API calls; a change triggers the full rebuild-recalibrate-update cycle automatically.*

### How to wire it in

Add two lines to the top of `run_pipeline.py` and any script that uses the RAG pipeline:

```
from kb_change_detector import check_and_recalibrate
check_and_recalibrate()
```

The fast path (hash match) completes in under a millisecond. The rebuild path runs only when content actually changed — which also means calibration queries run on the updated store, not the stale one, so the threshold always reflects the current knowledge base. The updated threshold is written to `config.py` automatically without any manual intervention.

> **For real client work:** when a client says "I added three new articles to the coaching library, can you update the bot?" — this script handles it automatically. Drop the new files into `knowledge_base/`, run the pipeline, and the change detector handles rebuilding, recalibrating, and updating the threshold. No manual steps.

### Testing results — all scenarios verified

| Scenario | Expected | Verified |
|---|---|---|
| First run, no hash on record | Triggers rebuild + calibration, saves hash | positive floor 0.503, threshold 0.302, hash saved |
| Second run, nothing changed | Fast path, zero API calls | Hash matched instantly, "unchanged" printed, 0 cost |
| File added to knowledge base | Detects change, rebuilds, recalibrates | New 4-file hash, store rebuilt, threshold updated |
| File deleted from knowledge base | Detects change, rebuilds, recalibrates | Back to 3-file hash, correct threshold restored |
| Safety floor activation (real data) | Safety floor overrides 60% heuristic | Test doc contaminated scores; floor set 0.05, correctly flagged |
| Cosine bug in force-rebuild path | Correct cosine scores after rebuild | Bug found during testing, fixed — both code paths now set cosine |

## 8. Silent audit logging

In health and wellness, if a client claims the AI gave dangerous or incorrect advice, you need an indisputable audit trail. `pipeline_logger.py` writes every significant pipeline event to a hidden internal file (`.pipeline_audit.log`) that is completely invisible to users and readable only by the operator. The client experience is entirely unchanged. The forensic record is always there.

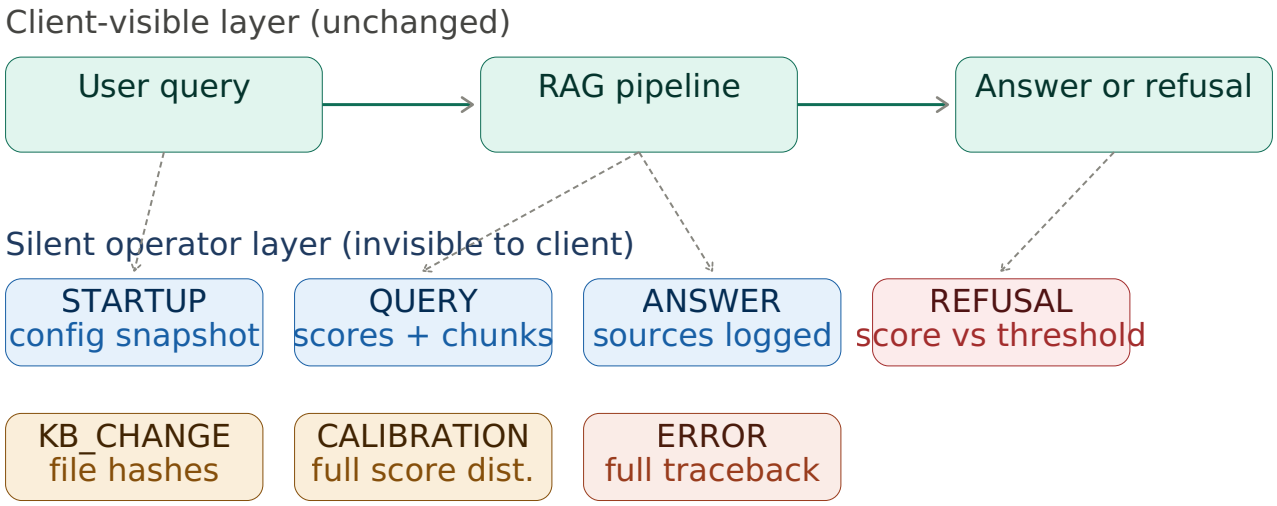



*Figure 8 — Silent audit logging. The client sees only the top layer. Every event is silently mirrored to the operator's log.*

### The seven event types and why each matters

| Event | What is recorded | Forensic value |
|---|---|---|
| **STARTUP** | Config snapshot: model, MIN_RELEVANCE, TOP_K, KB files with sizes. | If behaviour changed, verify whether config was different that day. |
| **QUERY** | Every user query with top 4 retrieved chunks and their relevance scores. | If the AI gave dangerous advice, see exactly which source chunks it was handed. |
| **ANSWER** | Generated answer (truncated to 500 chars) with source documents cited. | Full input-output audit trail for any given interaction. |
| **REFUSAL** | Query, exact score, threshold, and gap to threshold. | If a client claims valid questions were refused: show the score was 0.096 vs a 0.302 threshold — mathematically justified. |
| **KB_CHANGE** | Before/after file hashes, files added/ removed, timestamp. | "The AI changed overnight" — cross-reference this log. If a document was added the night before, that is the explanation. |
| **CALIBRATION** | Full score distribution: positive floor, negative ceiling, gap, recommended threshold, driver formula. | Every threshold change is documented with the evidence that justified it. |
| **ERROR** | Exception type, message, full traceback. | Incident investigation without needing to reproduce the failure. |

### Reading the log

The log file (`.pipeline_audit.log`) is never shown to clients. To read it as the operator:

```
python -c "from pipeline_logger import tail_log; tail_log(20)"
python -c "from pipeline_logger import tail_log; tail_log(50, 'REFUSAL')"
python -c "from pipeline_logger import tail_log; tail_log(10, 'KB_CHANGE')"
```

The log rotates automatically at 10MB (keeping 5 backups), uses structured JSON Lines format (one event per line, fully parseable), and never crashes the pipeline — all logging calls are exception-safe.

> **Verified output — all seven event types confirmed:**
>
> ```
> 2026-06-23 07:59:44 [STARTUP ] FitCoach Pro min_relevance=0.302 live=True
> 2026-06-23 08:00:02 [QUERY ] FitCoach Pro score=0.574 q=How much creatine...
> 2026-06-23 08:00:05 [ANSWER ] FitCoach Pro sources=['recovery_longevity.txt', ...]
> 2026-06-23 08:00:05 [REFUSAL ] FitCoach Pro score=0.096 threshold=0.302
> 2026-06-23 08:00:23 [KB_CHANGE ] FitCoach Pro added=[] removed=[]
> 2026-06-23 08:00:34 [CALIBRATION ] FitCoach Pro recommended=0.302 gap=0.332
> ```

## Appendix — production issues found & resolved

This pipeline was hardened through real deployment debugging. Each issue below is a citable, senior-level diagnosis. Total: 13 real production issues found and fixed.

| Issue | Root cause & fix |
|---|---|
| Wrong relevance scores on deploy | ChromaDB defaulted to L2 distance, not cosine — same content scored 0.08 deployed vs 0.54 locally. Fixed by explicitly setting cosine and pinning the version. |
| Stale vector cache | An old broken index from a failed deploy was being reused. Fixed by force-rebuilding the store on every startup. |
| App stuck "starting" | Gradio 6's SSR Node-proxy hung in the container. Fixed with `ssr_mode=False`. |
| Platform version conflict | The host force-installs its own Gradio; pinning a different version collided. Fixed by not pinning it and adapting the code. |
| Chunk topic-merging | Warm-up and injury content shared one chunk, diluting both embeddings. Fixed by splitting into single-topic chunks — context recall improved. |
| Cross-document gap | A fact spanning two documents wasn't synthesized well. Fixed by adding a bridging passage rather than over-tuning retrieval. |
| Diluted safety warning | A kidney-disease caution was softened by surrounding positive content. Fixed by expanding the caution so it carried more retrieval weight. |
| Overfitting (caught & reverted) | A relevance-gap filter fixed two cases but regressed two others. Correctly reverted — a discipline signal, not a failure. |
| Hardcoded relevance threshold | The 0.25 cutoff was an arbitrary constant. Replaced with a calibration utility. Calibrated value: 0.302 (positive floor 0.503, negative ceiling 0.165, gap 0.338). |
| 60% heuristic vulnerability | Calibration formula silently failed when negative ceiling was high. Fixed with max(negative_ceiling + 0.05, min_positive × 0.60). All four scenarios verified. |
| Non-UTF-8 file encoding crash | PowerShell echo writes UTF-16 by default, crashing the pipeline. Fixed with utf-8-sig fallback — warns instead of crashing. |
| Cosine bug in force-rebuild path | create_collection() in the force-rebuild path was missing cosine metadata. Produced near-zero scores (0.006 vs expected 0.503). Found during testing, fixed in both code paths. |
| Stale threshold after failed rebuild | The cosine bug left config.py holding an incorrect threshold (0.05). Fixed by clearing the vector store and hash record, forcing a clean rebuild. |

> **Cost reality:** a full live pipeline run costs roughly `$0.0007` . Embedding a client's content plus running the full audit stays well under one dollar, for a gross margin above 99%. The change detector adds roughly `$0.002` per recalibration run. Audit logging adds zero cost — it writes to local disk only.